

Revisão de Véspera - CI Inovador 2026

Sistemas Digitais + Inglês Instrumental

Objetivo: uma única leitura de 25 a 35 minutos no dia anterior

Conteúdo: somente regras, fórmulas e pegadinhas de maior retorno

Base: aulas 1-36, prova anterior, edital e erros pessoais

Não reescreva este arquivo. Leia um bloco, feche-o e tente explicar as regras principais. Marque no máximo cinco pontos que ainda mereçam uma última consulta.

1. Estratégia e regras da prova

Três passadas

1ª passada - diretas
marque: ✓ resolvida | ? entre duas | ! não identifiquei

2ª passada - dúvidas, contas e rastreios

3ª passada - procurar erros evitáveis

Na terceira passada, confira:

incorreta / exceto / principal / diretamente
signed versus unsigned
carry versus overflow
bit versus byte
ns versus ps
endereço versus conteúdo
estado atual versus próximo estado
stall versus flush
tema principal versus detalhe

Regra de troca:

Só mude uma resposta se surgir fórmula, trecho, tabela ou rastreio que prove a mudança.

Tempo sugerido

0-80 min primeira passada
80-150 min segunda passada
150-195 min terceira passada
195-225 min folha definitiva e conferência
225-240 min margem

Edital

portões abrem 13h30
portões fecham 14h
prova: 14h às 18h

Leve documento original com foto e caneta esferográfica preta. Não use relógio, calculadora, lápis ou borracha. Caderno e folhas-resposta são insubstituíveis. Rasura, marca incompleta, duas opções ou nenhuma opção tornam a marcação indevida.

pontuação = número de acertos
mínimo = 20/40
zerar uma disciplina elimina

Não há desconto por erro: responda todas e entregue o caderno e a folha definitiva.

2. Inglês instrumental

Protocolo

1. Leia primeiro o comando.
2. Classifique: tema, detalhe, vocabulário, conectivo ou objetivo.
3. Formule uma resposta antes de olhar as alternativas.
4. Volte ao trecho e procure evidência.
5. Elimine a alternativa verdadeira, mas secundária.

Tema principal

Main topic precisa cobrir o texto inteiro. Use o teste:

Esta alternativa serviria como título para todo o texto?

Não escolha uma opção apenas porque ela repete palavras. Paráfrase correta pode usar palavras diferentes e preservar sujeito, ação, intensidade e relação lógica.

Conectivos

Função	Palavras
Adição	moreover, furthermore, further, in addition
Consequência	therefore, thus, hence
Contraste	however, nevertheless, although
Exemplo	for example, such as
Resumo	in short, in summary
Brevidade	briefly = brevemente, em poucas palavras

avenues = caminhos, abordagens
 encompass = abranger
 fainter = mais tênue, menos intenso
 thus = portanto
 moreover = além disso

Marcadores de objetivo:

the aim is...
 this paper proposes...
 we investigate...
 this work presents...

May, might e could indicam possibilidade. Uma alternativa que transforma possibilidade em certeza costuma estar errada.

3. Binário, booleana e blocos combinacionais

Representação

1 hexadecimal = 4 bits
 unsigned n bits: 0 até $2^n - 1$
 C2 n bits: $-2^{(n-1)}$ até $2^{(n-1)} - 1$
 -X em C2: inverter bits e somar 1
 $A - B = A + C2(B)$

Carry final indica estouro **unsigned**. Overflow signed ocorre quando dois operandos de mesmo sinal produzem resultado de sinal oposto.

BCD: cada dígito decimal usa 4 bits
 válidos: 0000 a 1001
 correção da soma BCD: +0110 quando o dígito ultrapassa 9

12 em BCD é 0001 0010, não 1100.

Gray: consecutivos mudam um bit
 $G = B \text{ XOR } (B \gg 1)$
 '0' = 0x30 'A' = 0x41 'a' = 0x61
 valor 7 não é o caractere '7'

Portas e álgebra

XOR = $A'B + AB'$ - entradas diferentes
 XNOR = $AB + A'B'$ - entradas iguais
 precedência: NOT, AND, OR

$A + A' = 1$ $AA' = 0$
 $A + AB = A$ $A(A + B) = A$
 $AB + AB' = A$
 $A + BC = (A + B)(A + C)$
 $A + A'B = A + B$
 $(A + B)' = A'B'$
 $(AB)' = A' + B'$

SOP favorece NAND-NAND; POS favorece NOR-NOR.

Mintermos e Karnaugh

SOP = $\sum(\text{índices onde } F=1)$
 POS = $\prod(\text{índices onde } F=0)$

Mintermo: bit 1 direto e 0 negado. Maxtermo: regra inversa.

ordem Gray: 00,01,11,10
 grupos: 1,2,4,8,16
 bordas e cantos são adjacentes
 diagonal e grupo de 3 não valem
 variável que muda desaparece
 X pode ajudar ou ser ignorado

SOP agrupa 1; POS agrupa 0.

Blocos

MUX: muitos para um
 DEMUX: um para muitos
 decoder: código para linha
 encoder: linha para código
 MUX 2:1: $Y = S'I_0 + SI_1$
 n seletores escolhem 2^n entradas
 decoder n para 2^n
 encoder 2^n para n
 encoder decimal-BCD: 10 para 4; entradas simultâneas exigem prioridade

4. FSM, registradores, contadores e temporização

Estado e armazenamento

Moore: saída=f(estado)
 Mealy: saída=f(estado,entrada)
 N estados binários: $\text{ceil}(\log_2 N)$ FFs
 one-hot: N estados usam N FFs

Na síntese com flip-flop D:

$D = Q+$

D recebe o próximo estado desejado, não obrigatoriamente o estado atual.

latch: sensível a nível
 flip-flop: sensível à borda
 FF D: $Q+=D$
 FF T: $Q+=Q \text{ XOR } T$

PISO: paralelo entra, serial sai
 se a saída é Q_0 , Q_0 sai primeiro

ring: realimentação direta; n FFs, n estados
 Johnson: realimentação invertida; n FFs, 2n estados

Contadores

contador binário módulo M: mínimo $\text{ceil}(\log_2 M)$ FFs
 ripple/assíncrono: clock percorre FFs e acumula atraso
 síncrono: clock comum, mais rápido e previsível
 contador binário completo: $Q_i = \text{fclock} / 2^{(i+1)}$

Setup, hold e skew

Definindo skew = clock_destino - clock_origem:

setup:
 $T_{clk} \geq t_{CQmax} + t_{combmax} + t_{setup-skew}$

hold:
 $t_{CQmin} + t_{combmin} \geq t_{hold} + \text{skew}$

$F_{max}(MHz) = 1000 / T_{clk}(ns)$

setup: caminho máximo; dado chega tarde
 hold: caminho mínimo; dado muda cedo
 skew positivo: ajuda setup, piora hold
 skew negativo: piora setup, ajuda hold

Reduzir frequência pode ajudar setup, mas normalmente não corrige hold.

5. Aritmética, flags e ponto flutuante

Ponto fixo e IEEE 754

```
escala=2^F
valor_real=inteiro/2^F
resolução=2^-F
mais F: melhor resolução e menor faixa inteira
```

IEEE 754 simples:

```
1 sinal, 8 expoente, 23 fração, bias 127
normal=(-1)^s x 1.f x 2^(E-127)

E=255, fração=0 - infinito
E=255, fração!=0 - NaN
E=0, fração!=0 - subnormal
```

Somadores e subtração

```
Half-adder:
S=A XOR B
C=AB

Full-adder:
S=A XOR B XOR Cin
Cout=AB+A.Cin+B.Cin
```

Ripple-carry:

```
Cout de um estágio vai ao Cin seguinte
pior soma=(n-1)tcarry+tsoma
Cout final=n.tcarry
```

```
A-B=A+B'+1
SUB=0 soma
SUB=1 subtração
```

Na convenção ARM:

```
C=1 - sem borrow
C=0 - houve borrow
```

Overflow:

```
soma: mesmos sinais geram resultado de sinal oposto
A-B: A e B têm sinais diferentes e resultado difere de A
```

ULA e shifts

```
Z: resultado zero
N: bit mais significativo
C: carry/sem borrow, conforme a operação
V: overflow signed
```

```
LSL k: multiplica por 2^k, se não perder bits
LSR: entra zero
ASR: replica o sinal
```

Carry e overflow signed não são a mesma coisa.

6. CMOS, potência, atraso, memórias e FPGA

MOS e CMOS

NMOS liga com 1 - pull-down - saída 0
PMOS liga com 0 - pull-up - saída 1

Na rede pull-down:

NMOS série = AND das condições
NMOS paralelo = OR das condições
pull-down conduz para G
saída $Y=G'$

NAND: NMOS em série. NOR: NMOS em paralelo.

Potência e atraso

$P=VI$
 $E=Pt$
 P_{din} aproximadamente αCV^2f
 P_{est} aproximadamente $VDD \cdot I_{leak}$
 $E_{cap}=1/2 CV^2$
 E_{fonte} no 0 para $1=CV^2$

$1 \text{ pF} \cdot V^2 = 1 \text{ pJ}$
VDD dobra - P_{din} quadruplica
fanout e capacitância maiores - mais atraso e potência
caminho crítico - maior atraso

Glitch costuma resultar de atrasos diferentes em caminhos reconvergentes.

hazard elétrico: atraso/glitch
hazard de pipeline: conflito entre instruções

Memórias

n bits de endereço - 2^n posições
capacidade = posições x bits por palavra

registradores: muito rápidos e poucos
SRAM: rápida, sem refresh, volátil, usada em cache
DRAM: densa, com refresh, volátil, RAM principal
ROM: não volátil
PROM: normalmente programada uma vez

Lógica programável

PROM: AND fixo, OR programável
PLA: AND e OR programáveis
PAL: AND programável, OR fixo
LUT de k entradas: 2^k posições

FPGA contém LUTs, flip-flops e interconexões. FPGA baseado em SRAM perde a configuração ao desligar e recarrega um bitstream ao ligar.

7. Assembly e datapath

Notação

R1 = conteúdo do registrador
 [R1] = memória no endereço contido em R1
 #5 = imediato 5

 LDR/LOAD = memória para registrador
 STR/STORE = registrador para memória

Endereço efetivo:

base+offset = base+deslocamento
 indexado = base+índice x tamanho do elemento
 índice<<2 = índice x 4 para words de 32 bits

Offset normalmente está em bytes. Endianness muda a ordem dos bytes, não dos bits.

Comparação e branches

CMP A,B calcula A-B apenas para atualizar flags.

BEQ: Z=1
 BNE: Z=0

 signed:
 BLT: N!=V
 BGE: N=V

 unsigned:
 BLO: C=0
 BHS: C=1

BNE não compara sozinho; consulta flags anteriores.

Funções

BL alvo - chama e grava retorno em LR
 BX LR - retorna
 SP - topo da pilha
 PUSH - salva
 POP - restaura

Uma função que executa outro BL deve preservar LR.

Datapath e controle

datapath: movimenta, armazena e opera dados
 controle: emite sinais que comandam o datapath

LOAD:
 ULA calcula endereço; MemRead=1; RegWrite=1

 STORE:
 ULA calcula endereço; MemWrite=1; RegWrite=0

 BEQ:
 compara e pode mudar PC; não grava a subtração

8. Cache, pipeline, branch prediction e FPU

Cache e prefetch

hit: encontrado
miss: ausente
localidade temporal: reutilização
localidade espacial: endereços próximos

direta: uma linha possível
por conjunto: conjunto fixo, qualquer via
totalmente associativa: qualquer linha

offset=log2(bytes por bloco)
conjuntos=capacidade/(bloco x vias)
índice=log2(conjuntos)
tag=bits restantes

Prefetch busca antes da demanda:

next-line: próximo bloco
stream: sequência
stride: passo constante

Pode reduzir misses, mas também poluir a cache e gastar banda/energia.

cache: cópia rápida
prefetch: busca antecipada
memória virtual: traduz e isola endereços

Pipeline e hazards

IF - ID - EX - MEM - WB
ciclos ideais=k+n-1
CPI=ciclos/instruções

Pipeline melhora principalmente o throughput, não necessariamente a latência individual.

estrutural: conflito de recurso
dados: dependência
controle: próximo PC incerto

forwarding: encaminha resultado já disponível
stall: espera
bubble: espaço inválido, efeito de NOP
flush: descarta caminho errado

LOAD R1,[R2]
ADD R3,R1,R4

Load-use imediato costuma exigir 1 stall + forwarding. Diminuir a frequência não elimina a dependência.

Branch prediction

sempre tomado: estática
dinâmica: usa histórico
1 bit: repete o último resultado
2 bits: contador saturante com histerese
00/01 prevê não tomado
10/11 prevê tomado
BTB fornece alvo
RAS prevê retorno

Erro de predição provoca flush.

FPU

FP32=32 bits
FP64=64 bits
FPU executa operações e pode suportar várias precisões

soma/subtração: alinhar expoentes
multiplicação: somar expoentes e corrigir bias
divisão: subtrair expoentes e corrigir bias
depois: normalizar e arredondar

FPU trata zero, subnormal, infinito e NaN. FPU não é cache nem memória virtual.

9. Folha única de fórmulas

QUANTIDADE

n bits - 2^n combinações
 N estados binários - $\text{ceil}(\log_2 N)$ FFs
 one-hot - N FFs
 ring - n estados
 Johnson - $2n$ estados
 capacidade memória = $2^{\text{endereços}} \times \text{bits/palavra}$

TEMPORIZAÇÃO

$T_{\text{clk}} \geq t_{\text{CQmax}} + t_{\text{combmax}} + t_{\text{setup-skew}}$
 $t_{\text{CQmin}} + t_{\text{combmin}} \geq t_{\text{hold+skew}}$
 $F_{\text{max}}(\text{MHz}) = 1000 / T(\text{ns})$
 $\text{ripple MSB} = (n-1)t_{\text{carry}} + t_{\text{soma}}$

ARITMÉTICA

unsigned n bits: 0 a $2^n - 1$
 $C2$ n bits: $-2^{(n-1)}$ a $2^{(n-1)} - 1$
 $A - B = A + B' + 1$
 ARM: $C=1$ sem borrow; $C=0$ com borrow
 valor fixo = inteiro / 2^F
 resolução = 2^{-F}

ENERGIA

$P = VI$
 $E = Pt$
 P_{din} aproximadamente $\alpha CV^2 f$
 P_{est} aproximadamente $VDD \cdot I_{\text{leak}}$
 $E_{\text{cap}} = 1/2 CV^2$
 $E_{\text{fonte}} = CV^2$ no carregamento

CACHE E PIPELINE

miss rate = $1 - \text{hit rate}$
 conjuntos = capacidade / (bloco $\times$ vias)
 offset = $\log_2(\text{bytes/bloco})$
 índice = $\log_2(\text{conjuntos})$
 ciclos ideais = $k + n - 1$
 $\text{CPI} = \text{ciclos/instruções}$

UNIDADES

8 bits = 1 byte
 32 bits = 4 bytes
 1000 ps = 1 ns
 1 pF = 10^{-12} F
 1 fF = 10^{-15} F
 1 pF $\cdot V^2 = 1$ pJ

10. Suas pegadinhas finais

Leia esta lista na última revisão:

Carry final não é overflow signed.
 C=1 na subtração ARM significa sem borrow.
 D recebe Q+, não necessariamente Q atual.
 PISO: se a saída é Q0, Q0 sai primeiro.
 Ring tem n estados; Johnson tem 2n.
 Setup olha caminho máximo; hold olha mínimo.
 Skew positivo ajuda setup e piora hold.
 Reduzir frequência não costuma corrigir hold.
 Pull-down descreve quando a saída é zero.
 $E_{cap} = 1/2 CV^2$; a fonte fornece CV^2 .
 Glitch elétrico não é hazard de pipeline.
 SRAM e DRAM são voláteis.
 Cache não é prefetch nem memória virtual.
 Pipeline melhora vazão, não necessariamente latência.
 Forwarding não resolve todo load-use.
 Stall espera; flush descarta.
 Sempre tomado é predição estática.
 BTB fornece alvo; não decide a condição.
 CMP atualiza flags e não guarda a subtração.
 BNE consulta Z; não compara sozinho.
 [R1] é memória no endereço R1.
 <<2 multiplica por 4; não soma 2.
 BLT signed não é BLO unsigned.
 Na soma float, alinham-se os expoentes.
 Moreover/further adicionam; in short resume.
 Detalhe verdadeiro não substitui o tema principal.

Antes de entregar a prova

☐ respondi todas as questões
☐ conferi incorreta/exceto
☐ conferi signed/unsigned
☐ conferi unidades
☐ conferi endereço/conteúdo
☐ conferi tema principal/detalhe
☐ passei as respostas sem pular linha
☐ não deixei duas marcações

Na véspera, encerre depois desta leitura. Não tente aprender outro capítulo. Na prova, transforme cada cenário em uma pergunta curta e exija uma justificativa concreta para a alternativa escolhida.